GNN — Barcelona Metro Lost-and-Found

A heterogeneous Graph Convolutional Network that predicts where TMB passengers will pick up lost items, and a decision rule that uses that prediction to recommend storage stations. Trained on synthetic events; designed to be extended to real questionnaire data.

For a plain-language explanation of the model, start with **OVERVIEW.md**. For the TMB return-rate analysis and locker-network business case, see **docs/TMB_summary.md** (1-page) or **docs/TMB_return_rate_analysis.md** (full).

Folder layout

```
GNN/
├── README.md                          # this file
├── OVERVIEW.md                        # plain-language explanation of the GNN
│
├── docs/                              # written reports
│   ├── TMB_return_rate_analysis.md    # full analysis of 2025 TMB data + projections
│   ├── TMB_summary.md                 # 1-page summary
│   ├── TMB_summary.pdf                # rendered PDF
│   └── render_pdf.py                  # rebuild the PDF after edits
│
├── data/                              # input data
│   └── tmb_2025.xlsx                  # TMB 2025 lost-and-found extractOk
│
├── presentations/
│   └── LostAndFoundPresentation.pptx
│
├── artifacts/                         # generated outputs (kept in repo)
│   ├── data/                          # synthetic event corpora
│   ├── models/dev/                    # original v1-era model checkpoint
│   ├── models_v2/                     # canonical v2 ablation results
│   ├── coldstart/                     # cold-start GNN evaluation
│   ├── figures/                       # rendered PNGs
│   ├── logs/                          # captured stdout from training runs
│   ├── ml_baselines*.json             # MLP/LogReg/RF/kNN baseline metrics
│   ├── viz/                           # auxiliary visualisations
│   └── _v3_archived/                  # abandoned richer-synth experiment (kept as history)
│
├── v1_node_classification/            # earlier (homogeneous-GCN) approach, kept for reference
│
└── *.py                               # all source code at top level (see below)
```

Source files (all at top level — flat module structure)

Core model + data

File	Role
`model.py`	The HetGNN architecture (R-GCN-style heterogeneous message passing + DistMult-style scoring head)

File	Role
`train.py`	Training loop + evaluation (`evaluate` , `evaluate_baselines` , `_device`)
`graph_build.py`	Assembles the heterogeneous graph from a list of events
`events.py`	`FoundEvent` schema + load/save/split helpers
`contexts.py`	Context-bin vocabulary (item × hour-bucket × day-bucket)
`decision.py`	Storage decision rule — minimise expected metro distance from predicted pickup
`network.py`	Network graph, transfer-aware distance, zone classification
`metro.py`	Barcelona TMB + FGC line definitions and coordinates
`synth.py`	Synthetic event generator (passenger-profile based)

Evaluation and ablation scripts

File	Role
`ablate.py`	Full ablation suite: no-metro, no-ctx-mp, no-mp, only-metro, no-lost-edges, no-lost-input
`ablate_shared_relations.py`	R-GCN vs vanilla GCN comparison (per-relation vs shared weights)
`coldstart.py`	Cold-start station inductive evaluation (hold out 15% of stations)
`ml_baselines.py`	MLP, LogReg, RandomForest, kNN baselines on warm test set
`ml_baselines_coldstart.py`	MLP and LogReg on the cold-start split
`sweep_lambda.py`	Movement-cost decision-rule λ sweep (E[hops] vs operator move-cost tradeoff)

Visualisation / auxiliary

File	Role
`viz.py`	Per-event prediction visualisation on the metro map
`viz_aggregate.py`	Corpus-level aggregation visualisations
`viz_training.py`	Training-evolution multi-panel figure (t-SNE) + sample event prediction
`network_map.py`	Barcelona station coordinates + zone classification for plots
`aggregate.py`	Corpus-level demand aggregation (per-station argmax + prob-sum)
`demo.py`	Interactive CLI demo
`make_slides.py`	PowerPoint slide-deck generator

Entry points

Typical workflow:

```
# 1. Generate synthetic data
python3 synth.py --n 50000 --out artifacts/data/synth_v2.jsonl
```

```
# 2. Train and run the full ablation suite
python3 ablate.py               # outputs to artifacts/models_v2/

# 3. Run non-GNN ML baselines for comparison
python3 ml_baselines.py         # warm test set
python3 ml_baselines_coldstart.py  # cold-start test

# 4. Cold-start inductive evaluation
python3 coldstart.py            # outputs to artifacts/coldstart/

# 5. Decision-rule λ sweep (passenger vs operator cost)
python3 sweep_lambda.py

# 6. Visualisations
python3 viz_training.py         # writes artifacts/figures/

# 7. Rebuild the TMB summary PDF
cd docs && python3 render_pdf.py
```

Most scripts accept `ABLATE_DATA` and `ABLATE_OUT` environment variables to override input data and output directory paths.

Notes on the cleanup

- Source code (`.py` files) is intentionally kept flat at the top level so module imports (`from model import ...` , `from events import ...`) continue to work without packaging changes.
- Generated outputs in `artifacts/` are kept in the tree so the headline numbers can be reproduced without rerunning training.
- The `_v3_archived/` subfolder contains a richer-synth experiment that was tested and reverted; results stayed similar so we returned to the simpler v2 synth. Kept for historical reference.
- `v1_node_classification/` is an earlier homogeneous-GCN approach, kept for reference and replaced by the current heterogeneous model.